

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

The Banknote Authentication dataset was loaded using pandas library. We observe that the dataset consists of numerical features , there are 4 in number , they are variance,skewness,curtosis,entropy and the target class representing authentic and forged banknotes , 0 for authentic and 1 for forged.

First Five Samples

Table 1: First Five Samples of the Dataset

Index	Variance	Skewness	Curtosis	Entropy	Class
0	3.6216	8.6661	-2.8073	-0.4470	0
1	4.5459	8.1674	-2.4586	-1.4621	0
2	3.8660	-2.6383	1.9242	0.1065	0
3	3.4566	9.5228	-4.0112	-3.5944	0
4	0.3292	-4.4552	4.5718	-0.9888	0

Observation:

As told before we can tell that the dataset has 4 numerical features and one target class . each row is one bank note and this notes are used for classification.

Dataset Information

Table 2: Dataset Summary

Number of Samples	1372
Number of Features	4
Target Classes	2 (Authentic, Forged)
Dataset Shape	(1372, 5)
Duplicate Rows	24
Missing Values	None

Observation:

The dataset has 1372 banknotes i.e., instances with no missing values , so there is no extra missing value handling. But the dataset contains 24 duplicate records.

Feature Data Types

Table 3: Data Types of Dataset Features

Feature	Data Type
Variance	Float64
Skewness	Float64
Curtosis	Float64
Entropy	Float64
Class	Int64

Observation:

All input features are observed to be continuous numerical values, buit the target labels of the class are represented as integers. Therefore no categorical encoding is required.

Descriptive Statistics

Table 4: Statistical Summary of the Dataset

Statistic	Variance	Skewness	Curtosis	Entropy	Class
Count	1372	1372	1372	1372	1372
Mean	0.4337	1.9224	1.3976	-1.1917	0.4446
Std Dev	2.8428	5.8690	4.3100	2.1010	0.4971
Minimum	-7.0421	-13.7731	-5.2861	-8.5482	0
25%	-1.7730	-1.7082	-1.5750	-2.4135	0
Median	0.4962	2.3197	0.6166	-0.5867	0
75%	2.8215	6.8146	3.1793	0.3948	1
Maximum	6.8248	12.9516	17.9274	2.4495	1

Observation:

By printing the descriptive statistics we can know the distribution, spread and range of the features. We observe that the features are having various ranges, which indicates that we have to normalize the features before training the perceptron.

Class Distribution

Table 5: Distribution of Target Classes

Class	Number of Samples
Authentic (0)	762
Forged (1)	610

Observation:

The data set is not completely balanced but it is nearly balanced as there are 762 authentic(0) banknotes and 610(1) forged banknotes.(The dataset is not much imbalanced)

Task 2: Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to understand the distribution of features, identify relationships among variables, detect outliers, and analyze the class distribution before training the perceptron model.

Feature Histograms

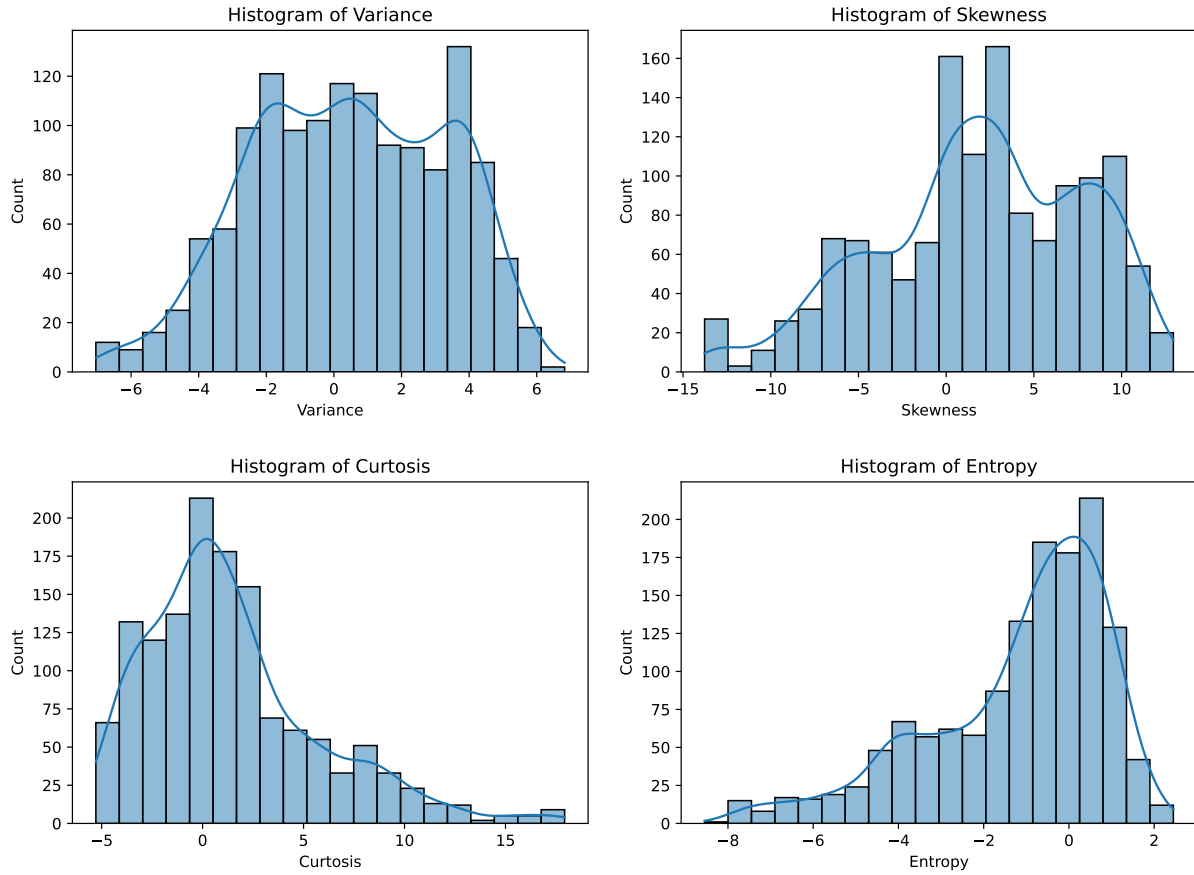


Figure 1: Histograms of the Numerical Features

Observation:

Histogram - Shows the distribution of each feature. The features are having different ranges and distributions, they are also having skewness and outliers.

Correlation Heatmap

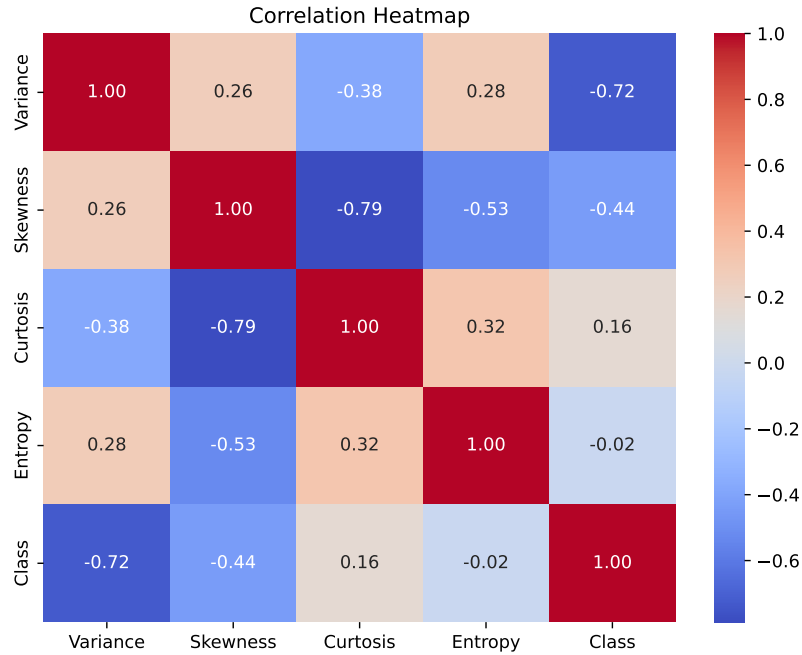


Figure 2: Correlation Heatmap

Observation:

The correlation heatmap - the relationships among the numerical features. Some features are weakly correlated and some other are moderately correlated.

Scatter Plots

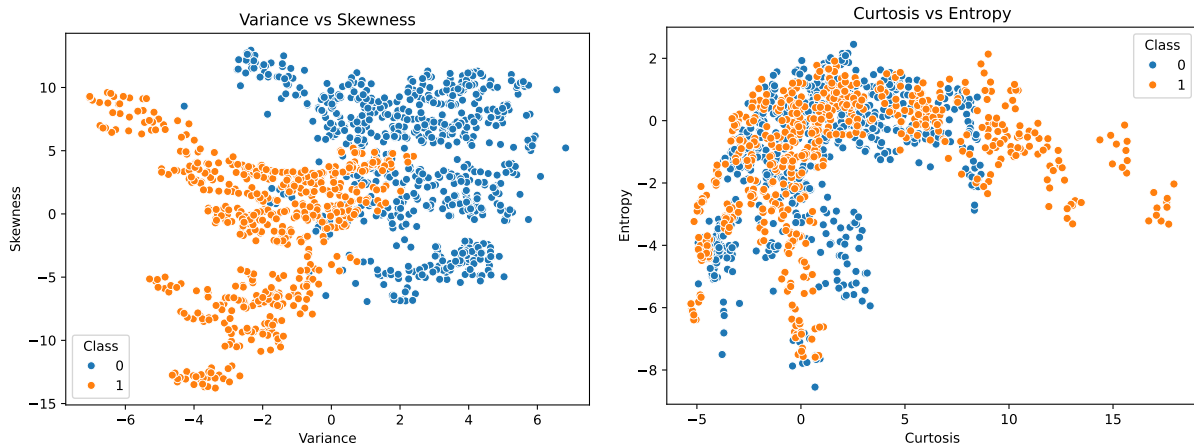


Figure 3: Scatter Plots of Selected Feature Pairs

Observation:

When we have plotted scatter plots they show that some of the authentic and forged banknotes are overlapped and also they occupy different regions in the feature space.

Pair Plot

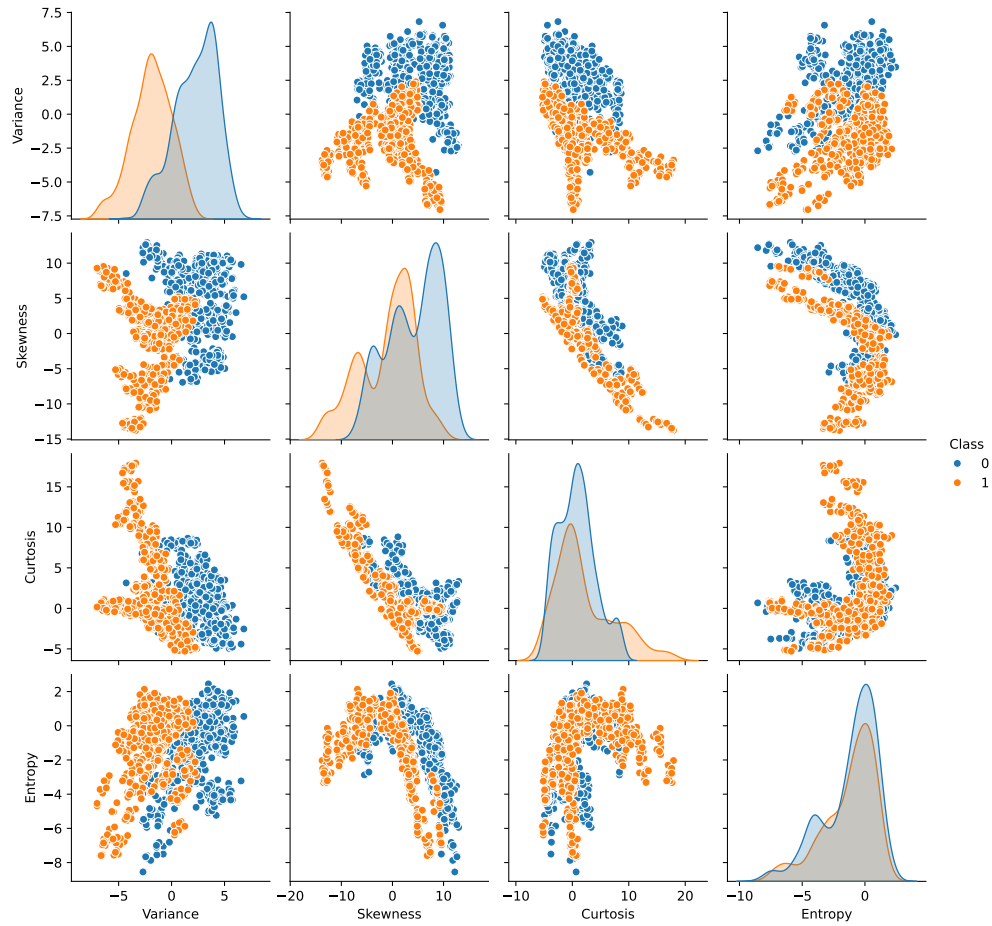


Figure 4: Pair Plot of the Dataset

Observation:

Pair plot - shows the relation ship between the features pair wise.

Boxplots

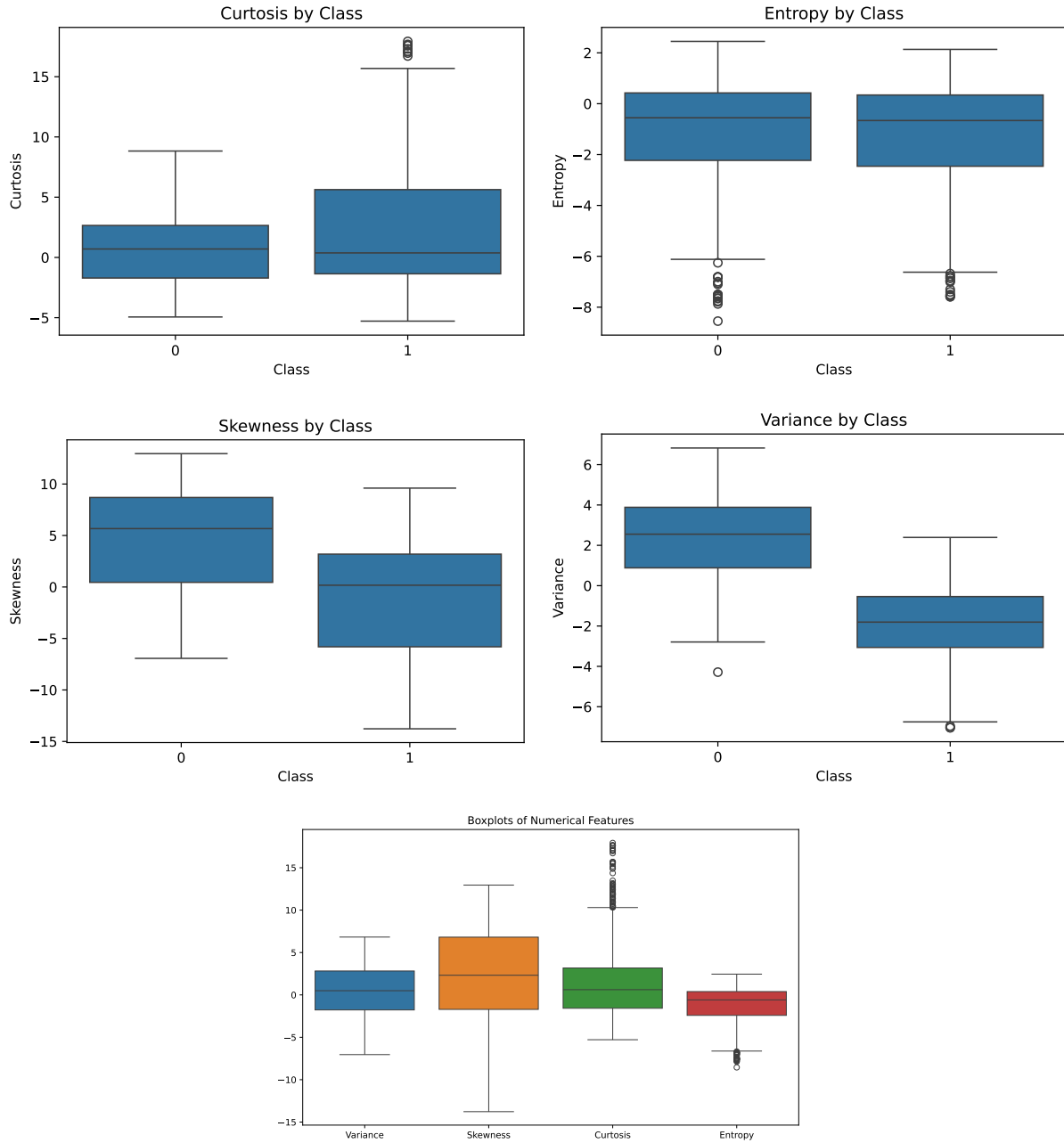


Figure 5: Boxplots of Dataset Features

Observation:

The boxplots show the spread, median, and presence of outliers in the dataset. The observed variations across features and classes tells that the selected features give some information for banknote classification.

Task 3: Data Preprocessing

The dataset was preprocessed before training the perceptron model. All numerical features were normalized using the StandardScaler technique so that each feature had a mean close to zero and a

standard deviation close to one. The normalized dataset was then divided into training and testing sets using an 80:20 ratio.

Normalized Features (First Five Samples)

Table 6: First Five Samples after Feature Normalization

Sample	Variance	Skewness	Curtosis	Entropy
1	1.1218	1.1495	-0.9760	0.3546
2	1.4471	1.0645	-0.8950	-0.1288
3	1.2078	-0.7774	0.1222	0.6181
4	1.0637	1.2955	-1.2554	-1.1440
5	-0.0368	-1.0870	0.7367	0.0966

Observation:

The large values in a feature dominate them, so the values are normalized to a common scale, this avoids problems of dominant large values in the learning process. This ensures stable perceptron training.

Training and Testing Split

Table 7: Train-Test Split Summary

Parameter	Value
Training Samples	1097
Testing Samples	275
Training Feature Shape	(1097, 4)
Testing Feature Shape	(275, 4)
Training Ratio	80%
Testing Ratio	20%

Observation:

The dataset was divided into 80% training data and 20% testing data. The training set was used to train the perceptron model and learn the perceptron parameters, while the testing set was used to evaluate the classifier's performance on unseen data.

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function

- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

The Single Layer Perceptron was trained using the perceptron learning rule with a learning rate of 0.01 for 20 epochs. During each epoch, the number of misclassified samples was computed, and the weights and bias were updated accordingly until convergence.

Epoch-wise Training Summary

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	47	-0.0725	-0.0649	-0.0645	0.0002	-0.03
2	26	-0.0822	-0.0938	-0.0515	-0.0052	-0.03
3	34	-0.0947	-0.0914	-0.0891	-0.0096	-0.05
4	23	-0.1107	-0.0958	-0.1063	-0.0131	-0.04
5	27	-0.1310	-0.1218	-0.0911	-0.0088	-0.05
6	25	-0.1395	-0.1284	-0.1049	-0.0067	-0.04
7	19	-0.1353	-0.1404	-0.1053	-0.0052	-0.05
8	19	-0.1335	-0.1448	-0.1049	0.0013	-0.06
9	16	-0.1343	-0.1318	-0.1240	-0.0116	-0.06
10	26	-0.1317	-0.1558	-0.1260	0.0115	-0.06
11	25	-0.1343	-0.1633	-0.1234	0.0044	-0.07
12	26	-0.1348	-0.1721	-0.1296	0.0013	-0.07
13	21	-0.1411	-0.1780	-0.1297	0.0057	-0.08
14	21	-0.1403	-0.1776	-0.1474	0.0063	-0.07
15	22	-0.1483	-0.1904	-0.1364	0.0000	-0.07
16	16	-0.1642	-0.1758	-0.1395	-0.0126	-0.07
17	17	-0.1496	-0.1711	-0.1574	-0.0185	-0.08
18	15	-0.1508	-0.1937	-0.1427	0.0083	-0.07
19	15	-0.1495	-0.1743	-0.1640	0.0067	-0.08
20	17	-0.1564	-0.1747	-0.1552	-0.0260	-0.09

Observation:

We observe that the no. of misclassified samples are decreasing for each epoch which shows that the model is properly trained and successfully learned the decision boundary. The weight and bias updates are stabilizing for each epoch which shows the convergence of the learning algorithm.

Task 6: Model Evaluation

After training the Single Layer Perceptron, the model was evaluated using standard classification metrics on the testing dataset. The performance was measured using Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix.

Classification Metrics

Table 9: Performance Metrics of the Perceptron Classifier

Metric	Value
Accuracy	0.9636 (96.36%)
Precision	1.0000
Recall	0.9213
F1-Score	0.9590

Observation:

The perceptron achieved an accuracy of 96.36

Confusion Matrix

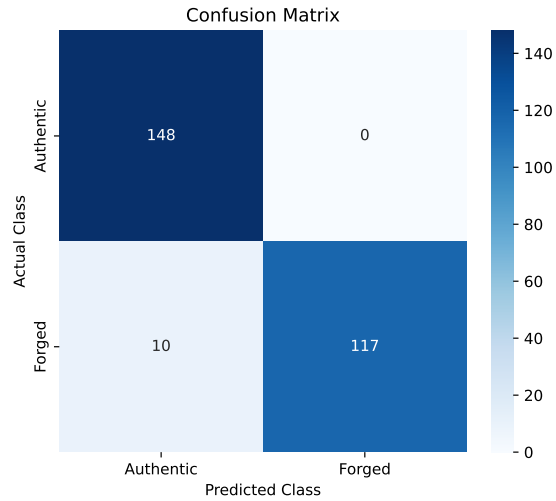


Figure 6: Confusion Matrix of the Perceptron Classifier

Confusion Matrix Values

	Predicted Authentic	Predicted Forged
Actual Authentic	148	0
Actual Forged	10	117

Observation:

The confusion matrix shows that the classifier correctly identified 148 authentic and 117 forged banknotes. Only 10 forged banknotes were misclassified as authentic, while no authentic banknotes were incorrectly classified as forged, demonstrating the robustness of the trained perceptron.

7. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.
Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

7.1 Feature Histograms

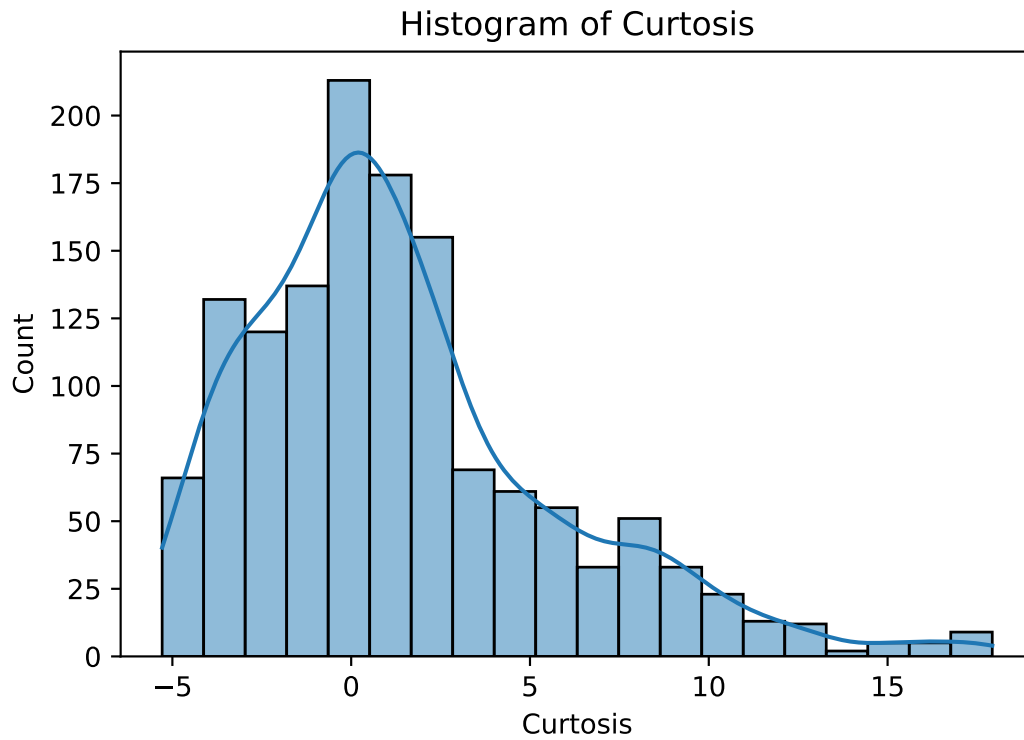


Figure 7: Feature Histograms

Observation: Histogram - Shows the distribution of each feature. The features are having different ranges and distributions , they are also having skewness and outliers.

7.2 Correlation Heatmap

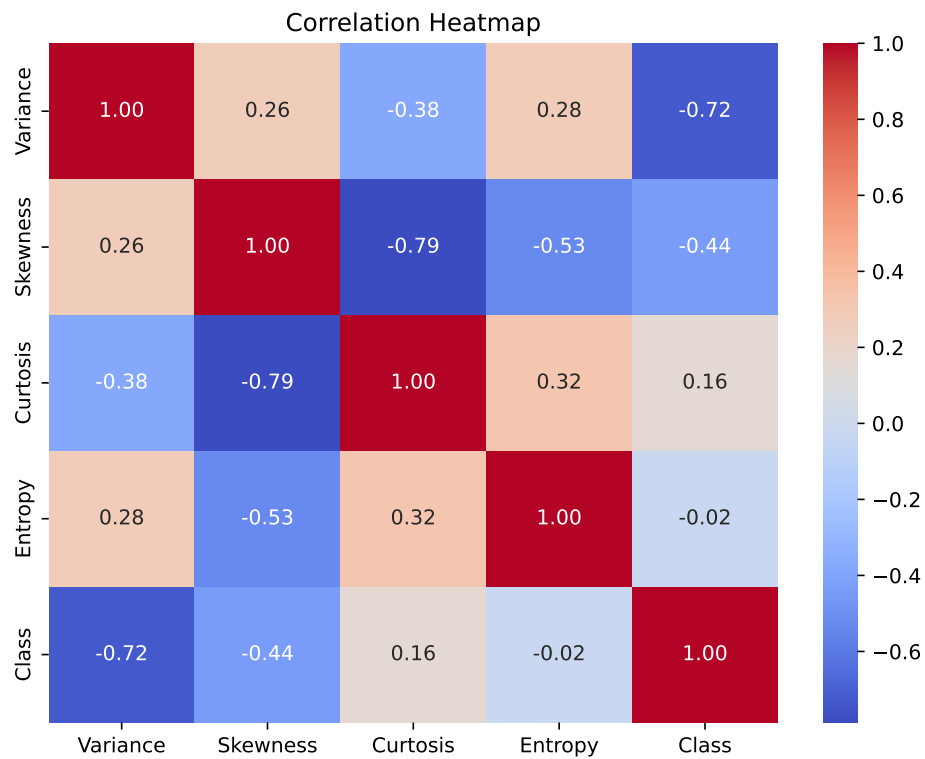


Figure 8: Correlation Heatmap

Observation: The correlation heatmap - the relationships among the numerical features. Some features are weakly correlated and some other are moderately correlated.

7.3 Scatter Plot

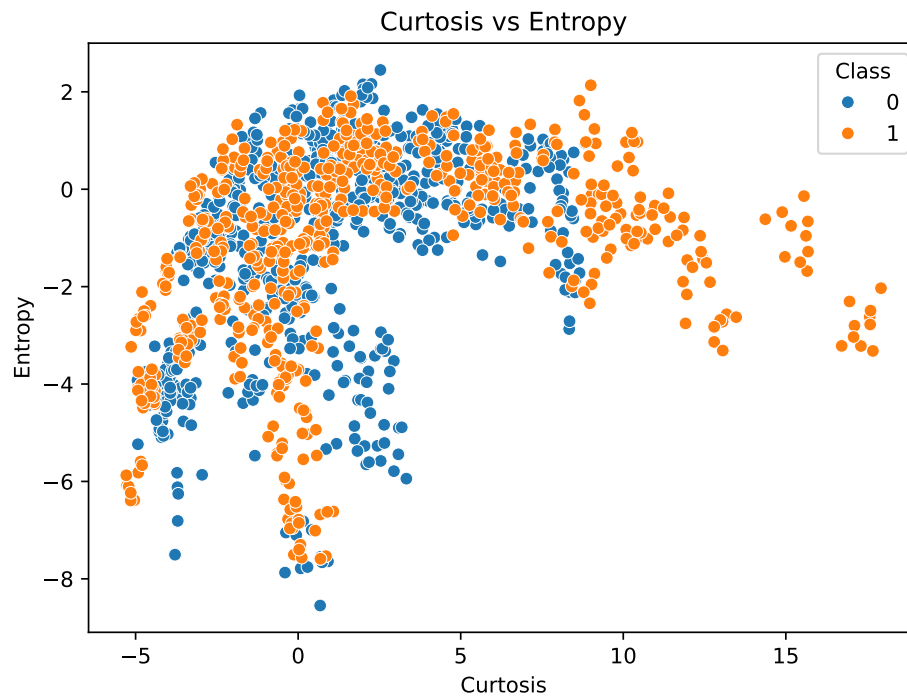


Figure 9: Scatter Plot

Observation: When we have plotted scatter plots they show that some of the authentic and forged banknotes are overlapped and also they occupy different regions in the feature space.

7.4 Training Error vs Epoch



Figure 10: Training Error vs Epoch

Observation: We observe that the number of misclassified samples are being decreasing for each epoch , this shows that the learning process of single layer perceptron is efficient and decision boundary is also learnt perfectly .

7.5 Weight Evolution

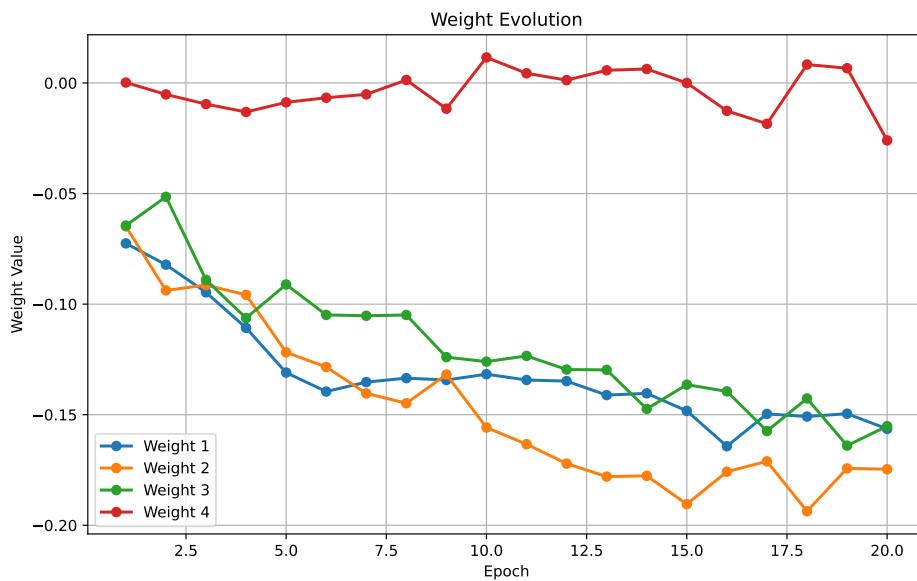


Figure 11: Weight Evolution

Observation: This plot shows that perceptron algorithm updates weights till convergence , we can clearly observe that the algorithm is trying to update the weights near to convergence at each epoch during featurte training. The weights are stabilizing as we increasing the number of epochs.

7.6 Bias Evolution

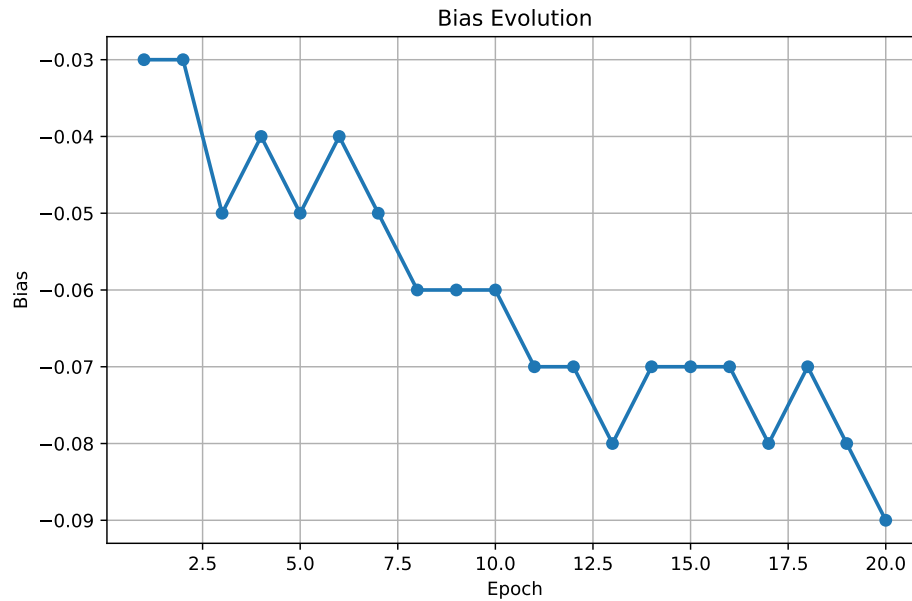


Figure 12: Bias Evolution

Observation: The bias terms are being updated at every epoch along with feature weights. same like weights the bias updation also is being stabilized for each epoch.

7.7 Confusion Matrix

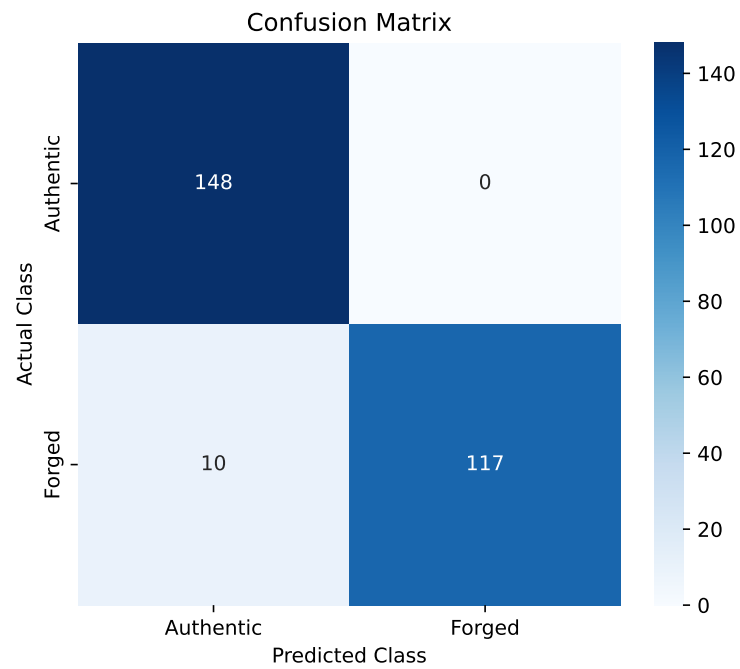


Figure 13: Confusion Matrix

Observation: The confusion matrix summarizes the classification performance of the perceptron on the test data. The high number of correctly classified samples and very few misclassifications indicate excellent classification performance.

7.8 Learning Rate Comparison

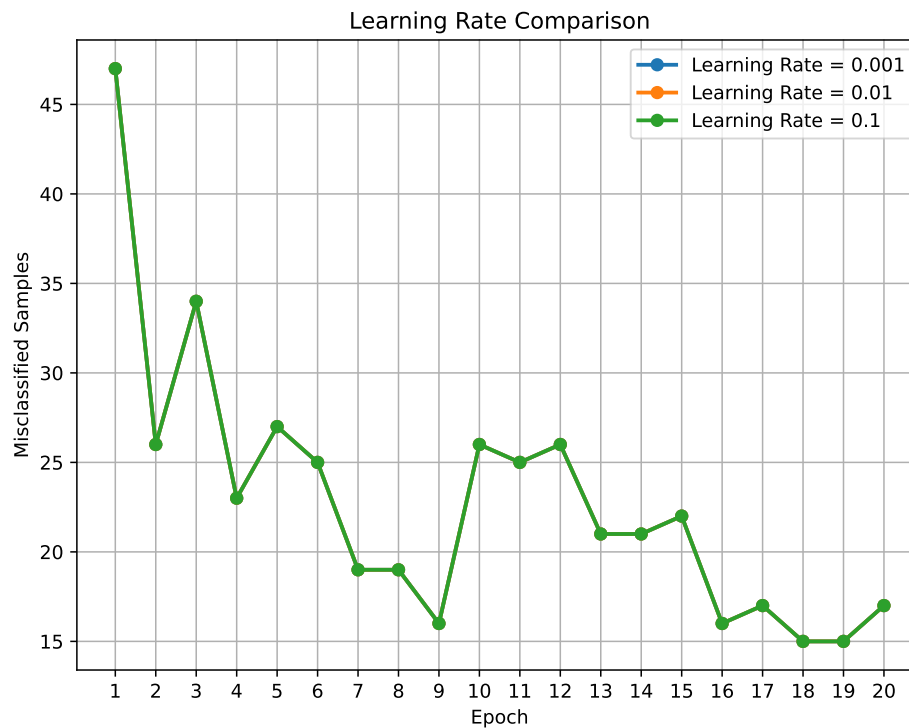


Figure 14: Learning Rate Comparison

Observation: When we plotted the learning rate comparison we see the change in magnitude of the weights updation, but the overall learning rate comparison is identical for all the three learning rates. The performance is identical for the 3 learning rates.

7.9 Decision Boundary (Optional)

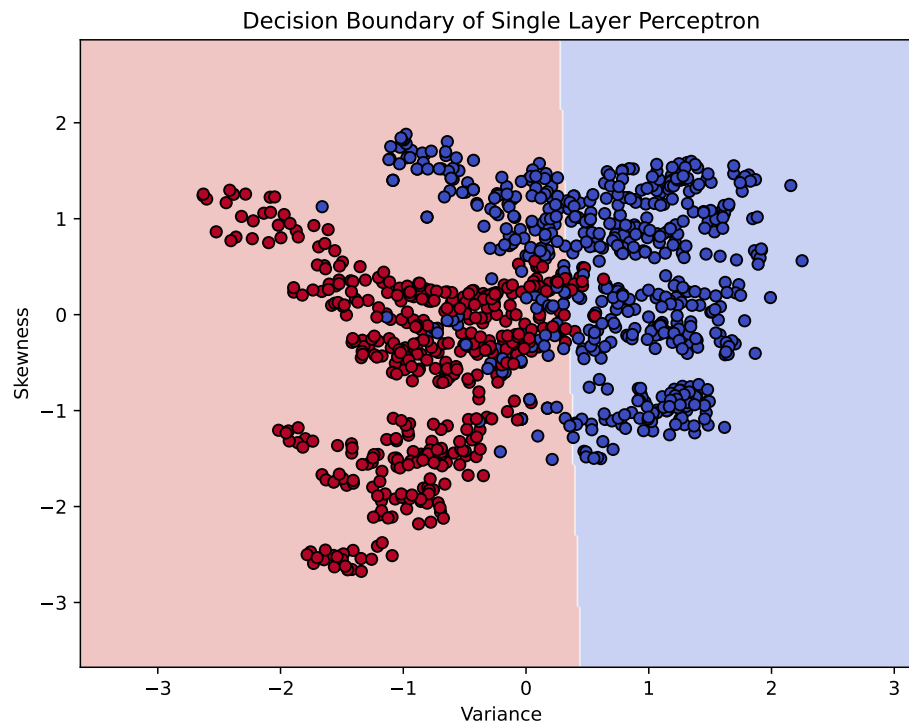


Figure 15: Decision Boundary

Observation: The single layer perceptron learned the decision boundary from both 0 and 1 classes. Here the decision boundary is plotted for 2 selected features. The decision boundary represents the linear hyper plane learned by the perceptron.

8. Performance Tables

Training Summary

Metric	Value
Dataset Size	1372 samples (4 features + 1 target)
Train/Test Split	Training: 1097 samples, Testing: 275 samples
Learning Rate	0.01
Epochs	20
Final Weights	[-0.1564, -0.1747, -0.1552, -0.0260]
Final Bias	-0.0900
Accuracy	0.9636
Precision	1.0000
Recall	0.9213
F1-score	0.9590

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Weight 3	Weight 4	Bias
1	47	-0.0725	-0.0649	-0.0645	0.0002	-0.0300
2	26	-0.0822	-0.0938	-0.0515	-0.0052	-0.0300
3	34	-0.0947	-0.0914	-0.0891	-0.0096	-0.0500
4	23	-0.1107	-0.0958	-0.1063	-0.0131	-0.0400
5	27	-0.1310	-0.1218	-0.0911	-0.0088	-0.0500
6	25	-0.1395	-0.1284	-0.1049	-0.0067	-0.0400
7	19	-0.1353	-0.1404	-0.1053	-0.0052	-0.0500
8	19	-0.1335	-0.1448	-0.1049	0.0013	-0.0600
9	16	-0.1343	-0.1318	-0.1240	-0.0116	-0.0600
10	26	-0.1317	-0.1558	-0.1260	0.0115	-0.0600
11	25	-0.1343	-0.1633	-0.1234	0.0044	-0.0700
12	26	-0.1348	-0.1721	-0.1296	0.0013	-0.0700
13	21	-0.1411	-0.1780	-0.1297	0.0057	-0.0800
14	21	-0.1403	-0.1776	-0.1474	0.0063	-0.0700
15	22	-0.1483	-0.1904	-0.1364	0.0000	-0.0700
16	16	-0.1642	-0.1758	-0.1395	-0.0126	-0.0700
17	17	-0.1496	-0.1711	-0.1574	-0.0185	-0.0800
18	15	-0.1508	-0.1937	-0.1427	0.0083	-0.0700
19	15	-0.1495	-0.1743	-0.1640	0.0067	-0.0800
20	17	-0.1564	-0.1747	-0.1552	-0.0260	-0.0900

0.1 AND Gate

Truth Table

Input 1	Input 2	Output
0	0	0
0	1	0
1	0	0
1	1	1

Table 12: Truth Table for AND Gate

Console Output

Epoch 1

```
-----  
Update : 1  
Input : [0 0]  
Target : 0  
Prediction : 1  
Weights : [0. 0.]  
Bias : -0.1
```

```
-----  
Update : 2  
Input : [1 1]  
Target : 1  
Prediction : 0  
Weights : [0.1 0.1]  
Bias : 0.0
```

Epoch 2

```
-----  
Update : 3  
Input : [0 0]  
Target : 0  
Prediction : 1  
Weights : [0.1 0.1]  
Bias : -0.1
```

```
-----  
Update : 4  
Input : [0 1]  
Target : 0  
Prediction : 1
```

Weights : [0.1 0.]
Bias : -0.2

Update : 5
Input : [1 1]
Target : 1
Prediction : 0
Weights : [0.2 0.1]
Bias : -0.1

Epoch 3

Update : 6
Input : [0 1]
Target : 0
Prediction : 1
Weights : [0.2 0.]
Bias : -0.2

Update : 7
Input : [1 0]
Target : 0
Prediction : 1
Weights : [0.1 0.]
Bias : -0.30000000000000004

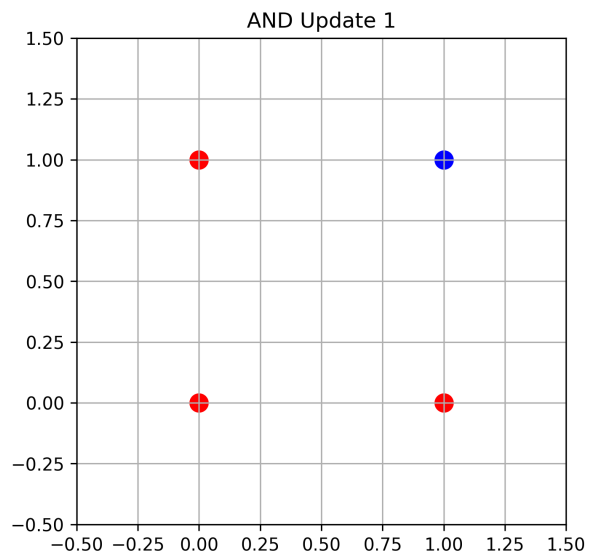
Update : 8
Input : [1 1]
Target : 1
Prediction : 0
Weights : [0.2 0.1]
Bias : -0.20000000000000004

Epoch 4

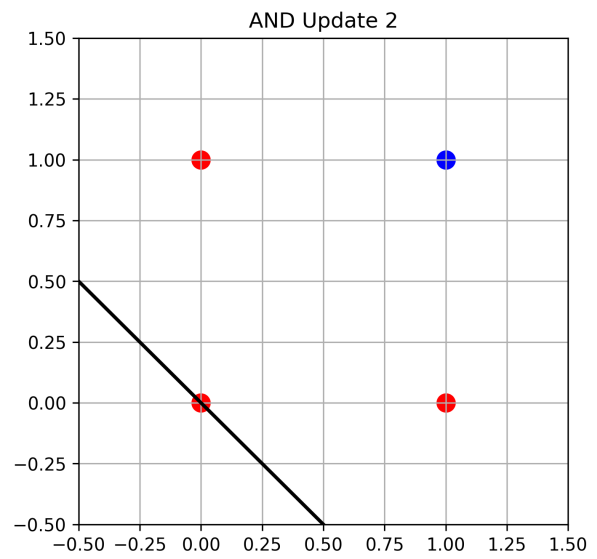
Training Converged

Final Weights : [0.2 0.1]
Final Bias : -0.20000000000000004

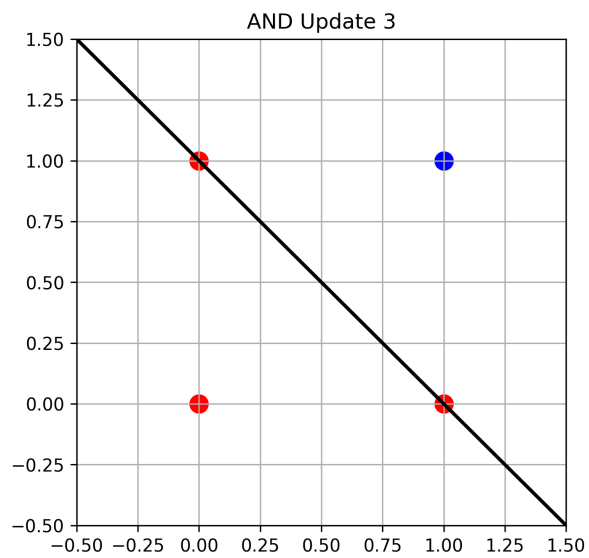
Decision Boundary after Updates



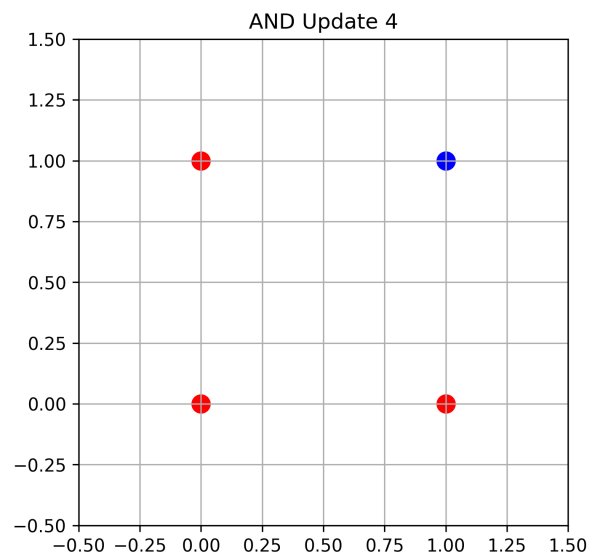
Update 1



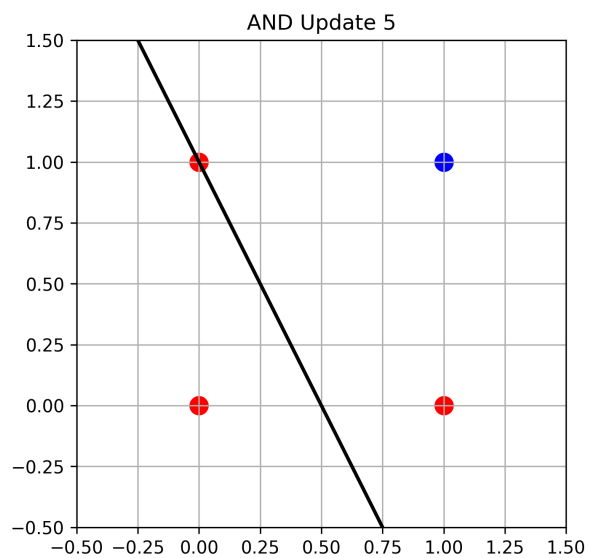
Update 2



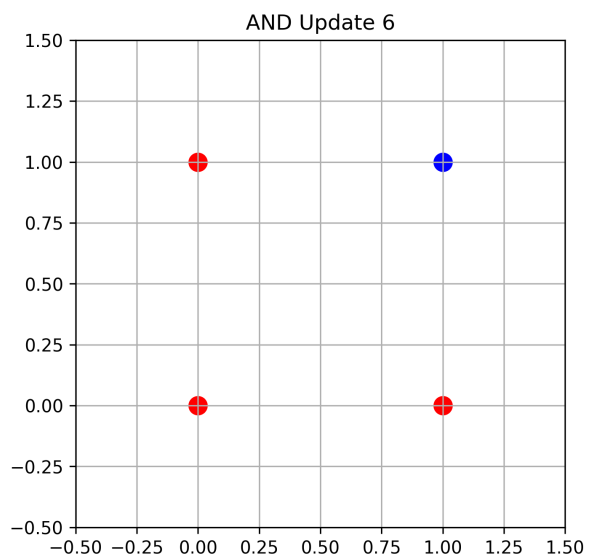
Update 3



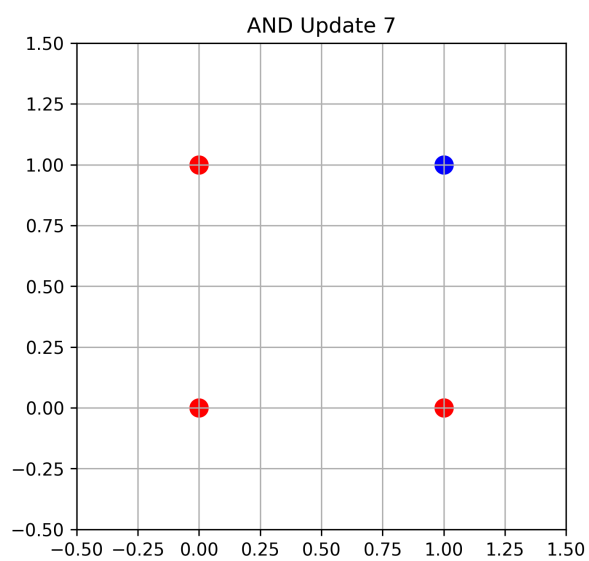
Update 4



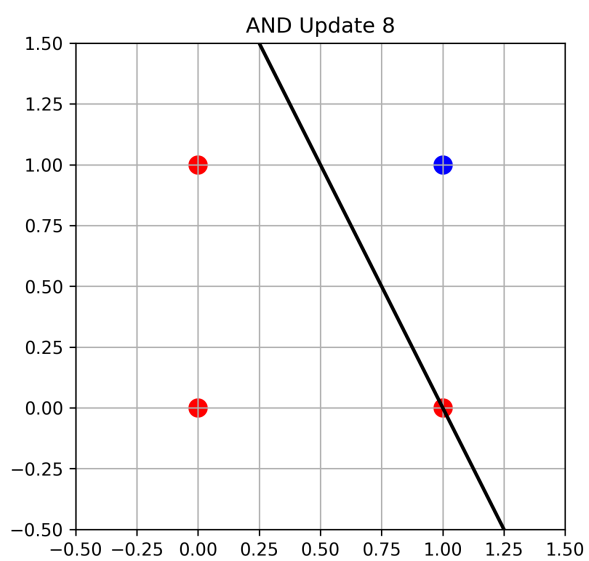
Update 5



Update 6



Update 7



Update 8

Final Decision Boundary

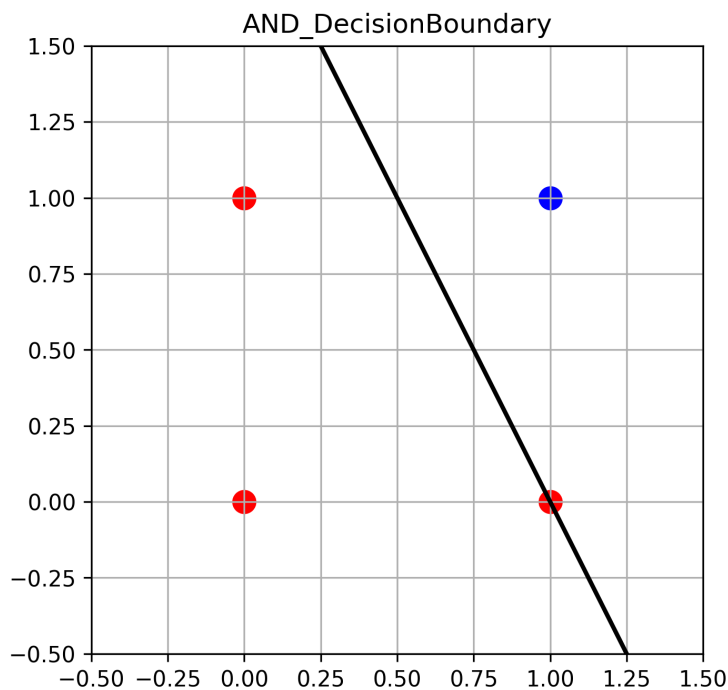


Figure 16: Final Decision Boundary for AND Gate

Observation

The Single Layer Perceptron learned the AND gate and after four epochs with eight weight updates. During the training, the weights and bias were adjusted whenever a sample was misclassified, this results in gradual improvement of the decision boundary. Since the AND gate is linearly separable, the single layer perceptron converged successfully and correctly classified all the four input combinations. The final weights obtained were $[0.2, 0.1]$ with a bias of -0.2 .

0.2 OR Gate

Truth Table

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	1

Table 13: Truth Table for OR Gate

Console Output

Epoch 1

Update : 1
Input : [0 0]
Target : 0
Prediction : 1
Weights : [0. 0.]
Bias : -0.1

Update : 2
Input : [0 1]
Target : 1
Prediction : 0
Weights : [0. 0.1]
Bias : 0.0

Epoch 2

Update : 3
Input : [0 0]
Target : 0
Prediction : 1
Weights : [0. 0.1]
Bias : -0.1

Update : 4
Input : [1 0]
Target : 1
Prediction : 0
Weights : [0.1 0.1]
Bias : 0.0

Epoch 3

Update : 5
Input : [0 0]
Target : 0
Prediction : 1
Weights : [0.1 0.1]
Bias : -0.1

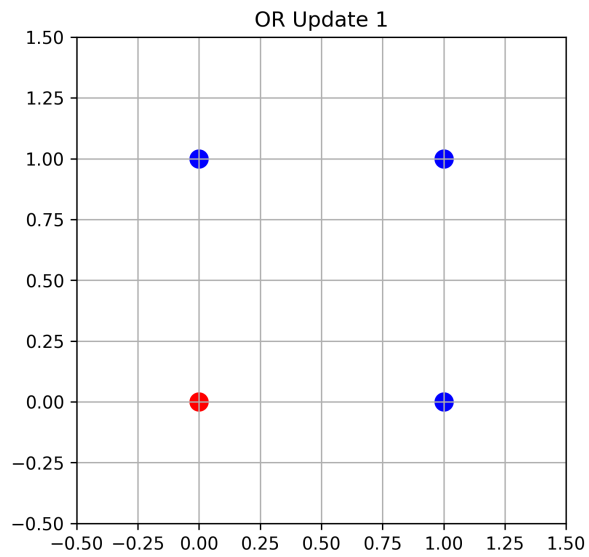
Epoch 4

Training Converged

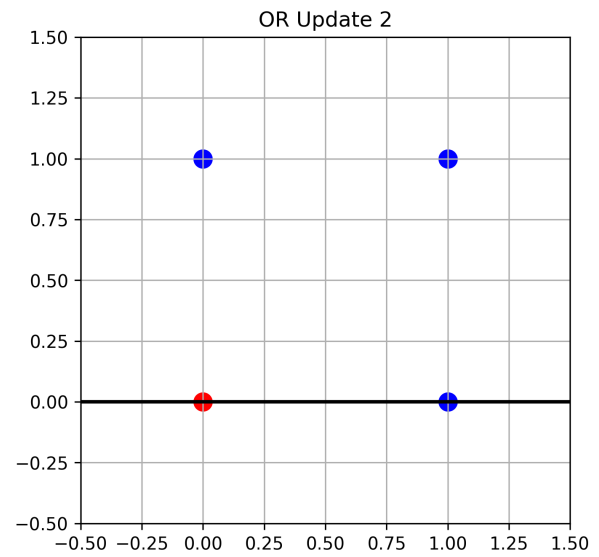
Final Weights : [0.1 0.1]

Final Bias : -0.1

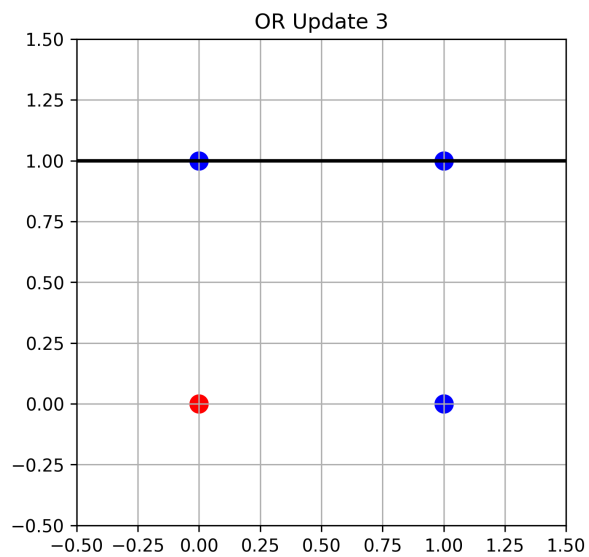
Decision Boundary after Updates



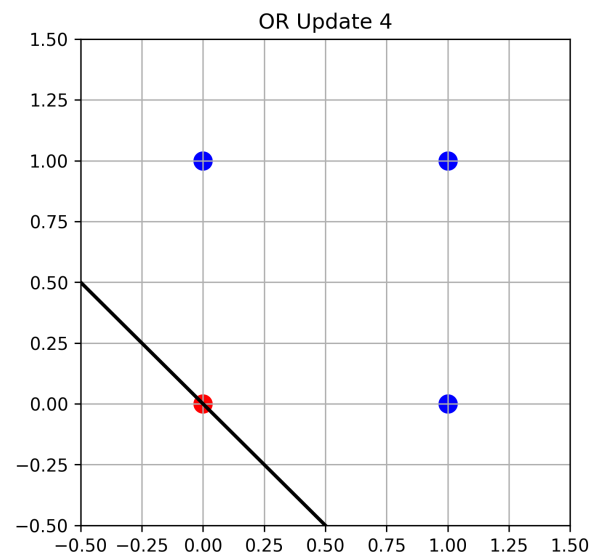
Update 1



Update 2



Update 3



Update 4

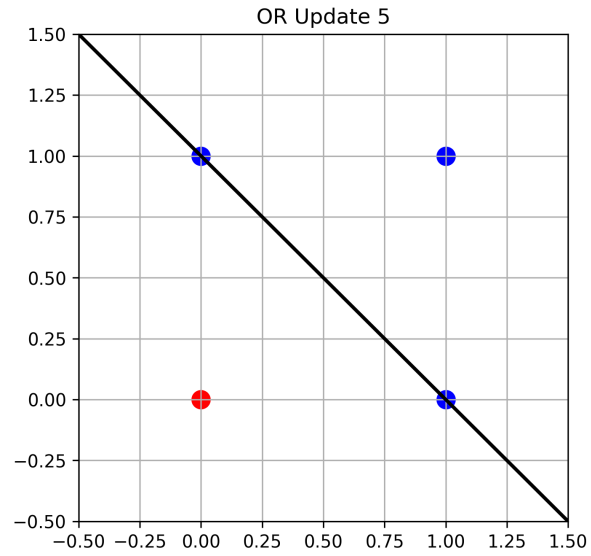


Figure 17: Decision Boundary after Update 5

Final Decision Boundary

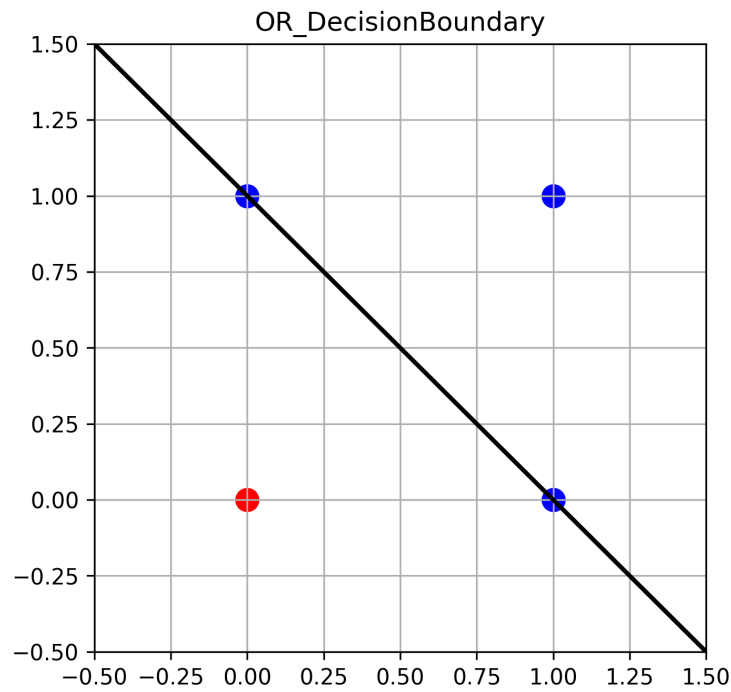


Figure 18: Final Decision Boundary for OR Gate

Observation

The Single Layer Perceptron learned the OR gate and after four epochs with five weight updates. During training, the weights and bias were modified whenever an incorrect prediction occurred, resulting the improvement of decision boundary. Since the OR gate is linearly separable, the single-

layer perceptron converged successfully and correctly classified all the four input combinations. The final weights obtained were $[0.1, 0.1]$ with a bias of -0.1 .

0.3 NOT Gate

Truth Table

Input	Output
0	1
1	0

Table 14: Truth Table for NOT Gate

Console Output

Epoch 1

```
-----  
Update : 1  
Input : [1 0]  
Target : 0  
Prediction : 1  
Weights : [-0.1  0. ]  
Bias : -0.1
```

Epoch 2

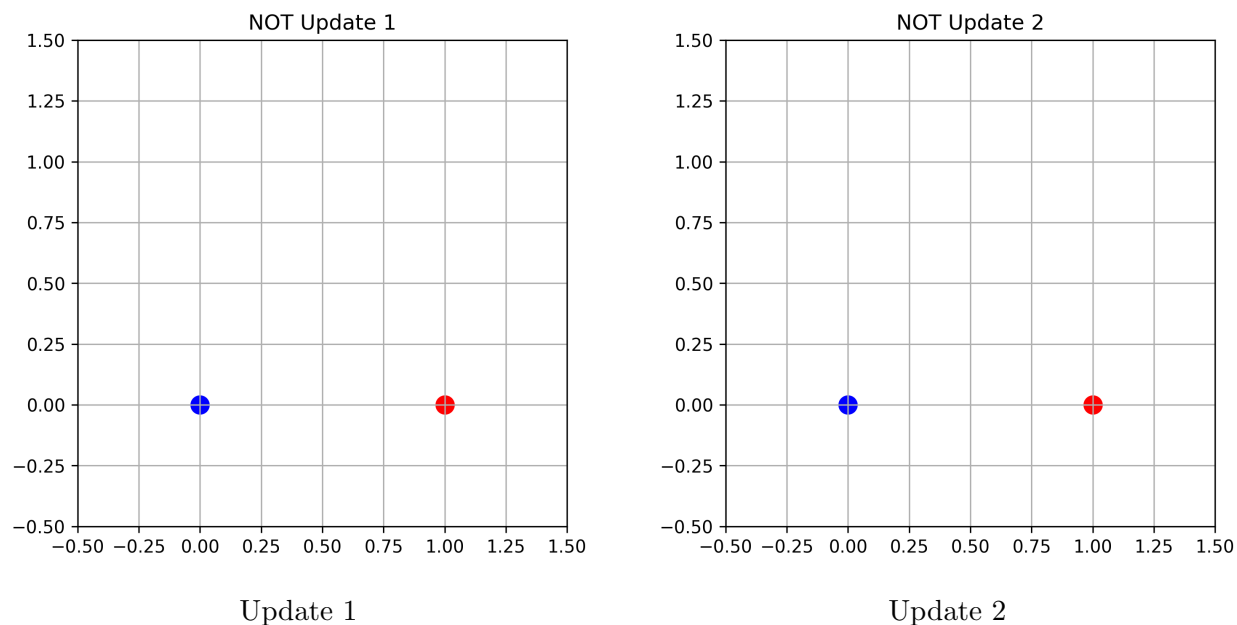
```
-----  
Update : 2  
Input : [0 0]  
Target : 1  
Prediction : 0  
Weights : [-0.1  0. ]  
Bias : 0.0
```

Epoch 3

Training Converged

Final Weights : [-0.1 0.]
Final Bias : 0.0

Decision Boundary after Updates



Observation

The Single Layer Perceptron learned the NOT gate after three epochs with only two weight updates. A two-dimensional input representation was used, where the second feature remained constant at zero, allowing visualization of the decision boundary while preserving the logical behavior of the NOT gate. Since the NOT gate is linearly separable, the single layer perceptron converged successfully and correctly classified both input patterns. The final weights obtained were $[-0.1, 0.0]$ with a bias of 0.0 .

0.4 XOR Gate

Truth Table

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Table 15: Truth Table for XOR Gate

Console Output

The Single Layer Perceptron continued updating its weights throughout the specified number of epochs without convergence.

A total of 38 weight updates were performed, after which the maximum number of epochs was reached.

Final Weights : [Your Final Weights]

Final Bias : [Your Final Bias]

Note: The complete epoch-wise console output consists of 38 updates and has been omitted, since the perceptron repeatedly updates the weights without convergence.

Final Decision Boundary

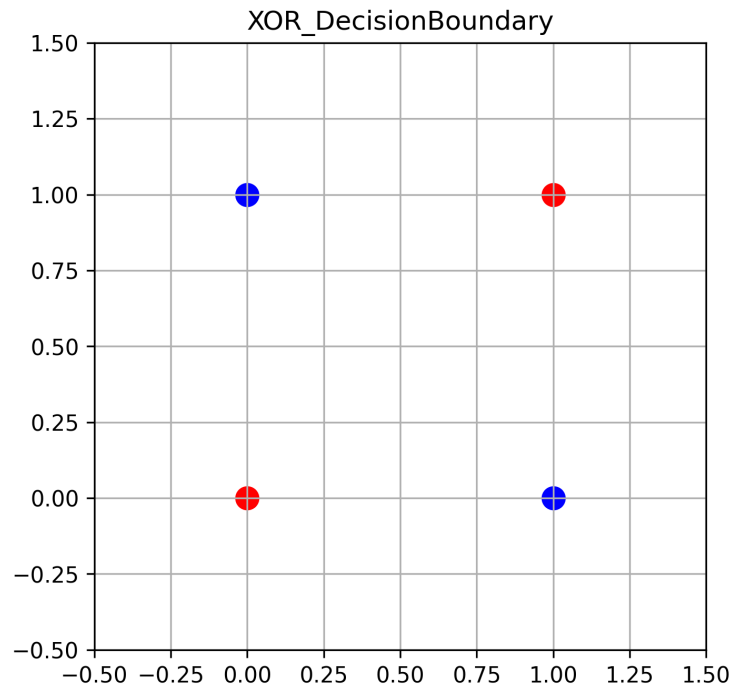


Figure 19: Final Decision Boundary for XOR Gate

Observation

Unlike the AND, OR, and NOT gates, the Single Layer Perceptron cannot learn the XOR gate. Even after 38 weight updates and the maximum number of training epochs, the algorithm failed to converge to a stable solution. This occurs because the XOR gate is not linearly separable therefore a single straight-line decision boundary cannot correctly classify all four input combinations. As a result, a Single Layer Perceptron is insufficient for solving the XOR problem, and a Multi-Layer Perceptron (MLP) with one or more hidden layers is required to correctly learn the XOR function.

9. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.

Observation:

The Step activation function produces only binary outputs (0 or 1), making it suitable for simple binary classification problems such as the Perceptron. but the step activation function is non-differentiable and has zero gradient almost everywhere, this prevents the use of gradient-based optimization methods like backpropagation. In the other hand, the Sigmoid activation function produces smooth outputs between 0 and 1 and it is differentiable, making the sigmoid activation function suitable for training modern neural networks.

2. Compare the implementation with Scikit-learn's Perceptron.

Table 16: Comparison of Custom Perceptron and Scikit-learn Perceptron

Parameter	Custom Perceptron	Scikit-learn Perceptron
Features Used	4	4
Learning Rate	0.01	0.01
Epochs	20	20
Accuracy	0.9636	0.9782
Weight 1	-0.1564	-0.1902
Weight 2	-0.1747	-0.1868
Weight 3	-0.1552	-0.1648
Weight 4	-0.0260	-0.0110
Bias	-0.0900	-0.0900

Observation:

Both the built-in and algorithm implementations were trained using the same four input features, learning rate (0.01), and 20 training epochs. We see that the Scikit-learn Perceptron achieved a slightly higher accuracy of **97.82%** compared to the algorithm's manual implementation of perceptron **96.36%**. The learned weights and bias are also very similar, indicating that the manual algorithm implementation correctly follows the perceptron learning algorithm and produces results comparable to the built-in Scikit-learn implementation.

3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.

Table 17: Learning Rate Comparison

Epoch	0.001	0.01	0.1
1	47	47	47
2	26	26	26
3	34	34	34
4	23	23	23
5	27	27	27
6	25	25	25
7	19	19	19
8	19	19	19
9	16	16	16
10	26	26	26
11	25	25	25
12	26	26	26
13	21	21	21
14	21	21	21
15	22	22	22
16	16	16	16
17	17	17	17
18	15	15	15
19	15	15	15
20	17	17	17

Observation:

We see that the weights magnitude is increased with the increase in the learning rate but the classification , convergence and performance is identical for all the three learning rates.

4. **Explain why a Single Layer Perceptron cannot solve the XOR problem.**

Observation:

The XOR gate problem is not linearly separable(it is said to be non-linear), meaning that no single straight line can separate the two output classes. Since a Single Layer Perceptron can only learn linear(straight-line) decision boundaries, the single-layer perceptron cannot correctly classify the XOR dataset. Solving XOR requires a Multi-Layer Perceptron with one or more hidden layers.

5. **Study the effect of feature normalization on model convergence.**

Observation:

Feature normalization scales all input features to a similar range which prevents the domination of larger feature values in the learning process. After normalization, the features are made to lie on the common scale and then we observe that the perceptron converged more smoothly, the weight updates became more stable, and the classifier improved its classification performance.

10. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code
- Experimental Results
- Generated Plots with Interpretation
- Performance Tables
- Discussion
- Conclusion
- References

Source Code

The complete implementation of the Single Layer Perceptron, including data preprocessing, model training, evaluation, and visualization, is available in the following Google Colab notebook:

https://colab.research.google.com/drive/13DYL-Qzk3DIsX5IhD6Ina14F6_WF0t8b?usp=sharing

The XOR and Basic logic gates (AND, OR, NOT) are also implemented using a single layer perceptron. https://colab.research.google.com/drive/1VXsz4psKSRtkTyUoS51NTtDEf1J_UMJr?usp=sharing

git Repository: https://github.com/LaasyaPriya-1238/DL_LAB.git

Conclusion

In this experiment, a Single Layer Perceptron was successfully implemented from scratch (manual algorithm implementation) and applied to the Banknote Authentication dataset for binary classification. After preprocessing and training, the model achieved an accuracy of **96.36%**, demonstrating its effectiveness in classifying authentic and forged banknotes. The experimental results,

performance metrics, and comparison with Scikit-learn’s Perceptron validated the correctness of the implementation and highlighted the importance of feature normalization and appropriate learning parameters for achieving good classification performance.

11. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.